Dependency in DBMS

- A dependency is a constraint that governs or defines the relationship between two or more attributes.
- In a database, it happens when information recorded in the same table uniquely determines other information stored in the same table.
- This may also be described as a relationship in which knowing the value of one attribute (or collection of attributes) in the same table tells you the value of another attribute (or set of attributes).
- It's critical to understand database dependencies since they serve as the foundation for database normalization.

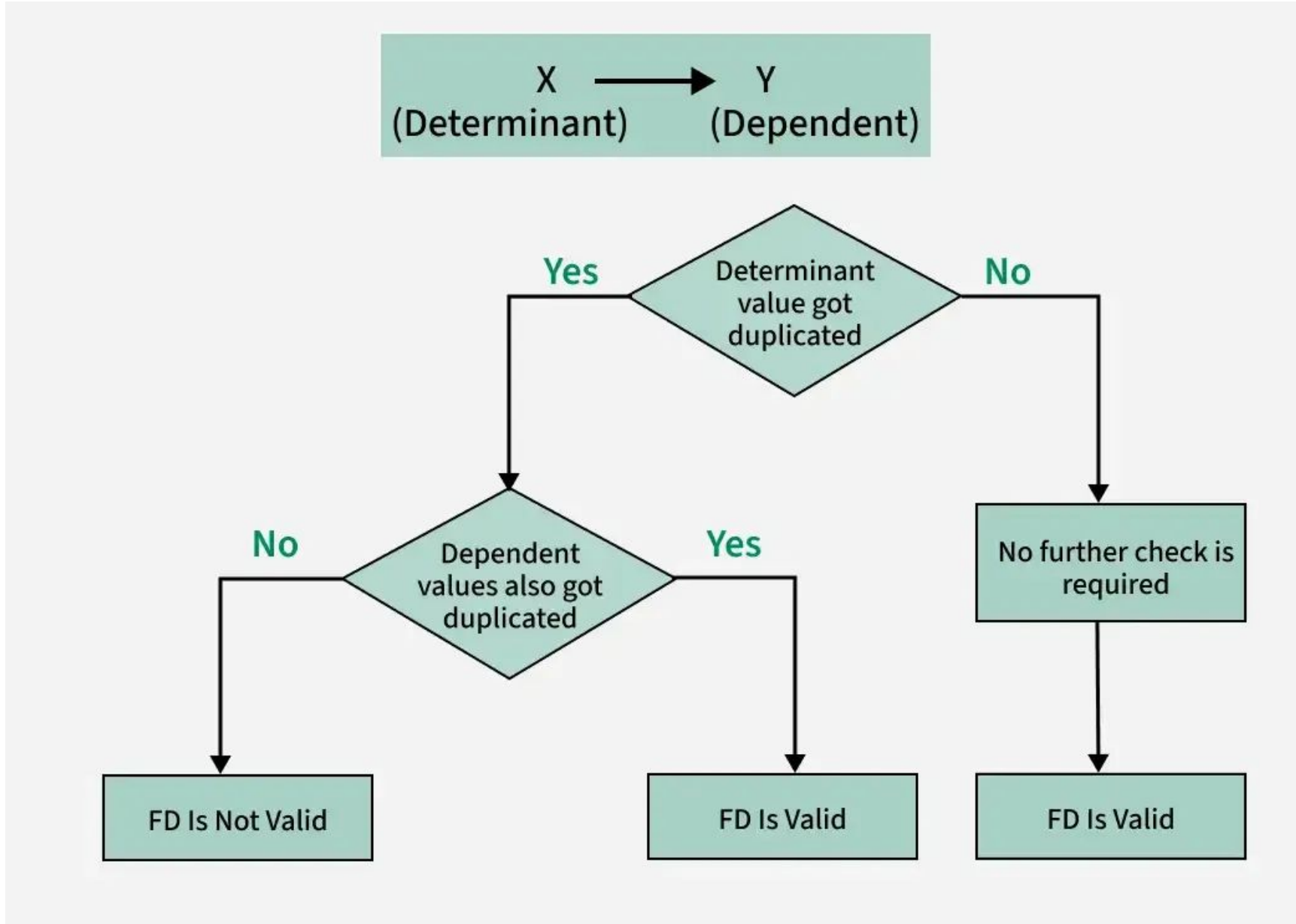
Functional Dependency

- A functional dependency occurs when the value of one attribute (or a set of attributes) uniquely determines the value of another attribute.
- This relationship is denoted as: **X** $\rightarrow$ **Y**
- Here, **X** is the determinant, and **Y** is the dependent attribute.
- This means that for each unique value of X, there is precisely one corresponding value of Y.

StudentID	StudentName	StudentAge
101	Rahul	23
102	Ankit	22
103	Aditya	22
104	Sahil	24
105	Ankit	23

StudentID -> StudentName
StudentID -> StudentAge

Note: The functional dependencies
StudentName -> StudentAge or
StudentAge -> StudentName
Do not hold.


X
(Determinant)
Y
(Dependent)
Determinant value got duplicated
Yes
No
Dependent values also got duplicated
No
Yes
No further check is required
FD Is Not Valid
FD Is Valid
FD Is Valid

- Functional dependency is expressed in the form of equations. For example, if we have an employee record with fields "EmployeeID", "FirstName" and "LastName" we can specify the function as follows:
- *EmployeeID -> FirstName, LastName*
- There are two main features to represent functional dependency in DBMS: left (LHS) and right (RHS) of the arrow (->).
- For example, if we have a table with attributes "X", "Y" and "Z" and the attribute "X" can determine the value of the attributes "Y" and "Z".
- *X -> Y, Z*
- This symbol indicates that the value in attributes "X" determines the values in attributes "Y" and "Z". So if you know the value of "X", you can also determine the value of "Y" and "Z".

- **Benefits of Functional Dependency in DBMS**
- The concept of normalization is based on functional dependencies.
- Using functional dependencies, a table is divided in multiple tables that helps us in preventing duplicate data and hence Improves Data Quality, less errors and better database design.

Types of Functional Dependency:

- Trivial Functional Dependency
- Non-Trivial Functional Dependency
- Multivalued Functional Dependency
- Transitive Functional Dependency
- Fully Functional Dependency
- Partial Functional Dependency

Types of Functional dependencies in DBMS

Trivial Functional Dependency

- In Trivial Functional Dependency, a dependent is always a subset of the determinant. i.e., If X → Y and Y is the subset of X, then it is called a trivial functional dependency.Symbolically: A→B is a trivial functional dependency if B is a subset of A. The following dependencies are also trivial: A→A & B→B

Example 1 :

- {ABC -> AB} {ABC -> A} {ABC -> ABC}

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Example 2: Here, {roll_no, name} → name is a trivial functional dependency, since the dependent name is a subset of determinant set {roll_no, name}.

Similarly, roll_no → roll_no is also an example of trivial functional dependency.

Non-trivial Functional Dependency

In Non-trivial functional dependency, the dependent is strictly not a subset of the determinant. i.e. If X → Y and Y is not a subset of X, then it is called Non-trivial functional dependency.

- **Example 1 :**
- Id -> Name
- Name -> DOB

Example 2:

Here, roll_no → name is a non-trivial functional dependency, since the dependent name is not a subset of determinant roll_no. Similarly, {roll_no, name} → age is also a non-trivial functional dependency, since age is not a subset of {roll_no, name}

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Multivalued Functional Dependency

- In Multivalued functional dependency, entities of the dependent set are not dependent on each other. i.e. If a → {b, c} and there exists no functional dependency between b and c, then it is called a multivalued functional dependency.

bike_model	color	accessory
tu1001	Black	Helmet
tu1001	Black	Gloves
tu1001	Red	Helmet
tu1001	Red	Gloves

In this table:

- X = bike_model
- Y = color
- Z = accessory

```
bike_model ↠ color
bike_model ↠ accessory
```

For each bike model (bike_model):

- There is a group of colors (color) and a group of accessories (accessory).
- The colors do not depend on the accessory, and the accessories do not depend on the colors. They are independent of each other.
- The sets of color and accessory are linked only to bike_model.

Transitive Functional Dependency

In transitive functional dependency, dependent is indirectly dependent on determinant. i.e. If a → b & b → c, then according to axiom of transitivity, a → c. This is a transitive functional dependency.

Example:

enrol_no	name	dept	building_no
42	abc	CO	4
43	pqr	EC	2
44	xyz	IT	1
45	abc	EC	2

Here, enrol_no → dept and dept → building_no.
Hence, according to the axiom of transitivity, enrol_no → building_no is a valid functional dependency.
This is an indirect functional dependency, hence called Transitive functional dependency.

Fully Functional Dependency

- In full functional dependency an attribute or a set of attributes uniquely determines another attribute or set of attributes. If a relation R has attributes X, Y, Z with the dependencies X->Y and X->Z which states that those dependencies are fully functional.

Partial Functional Dependency

- In partial functional dependency a non key attribute depends on a part of the composite key, rather than the whole key. If a relation R has attributes X, Y, Z where X and Y are the composite key and Z is non key attribute. Then X->Z is a partial functional dependency in RDBMS.

Normalization

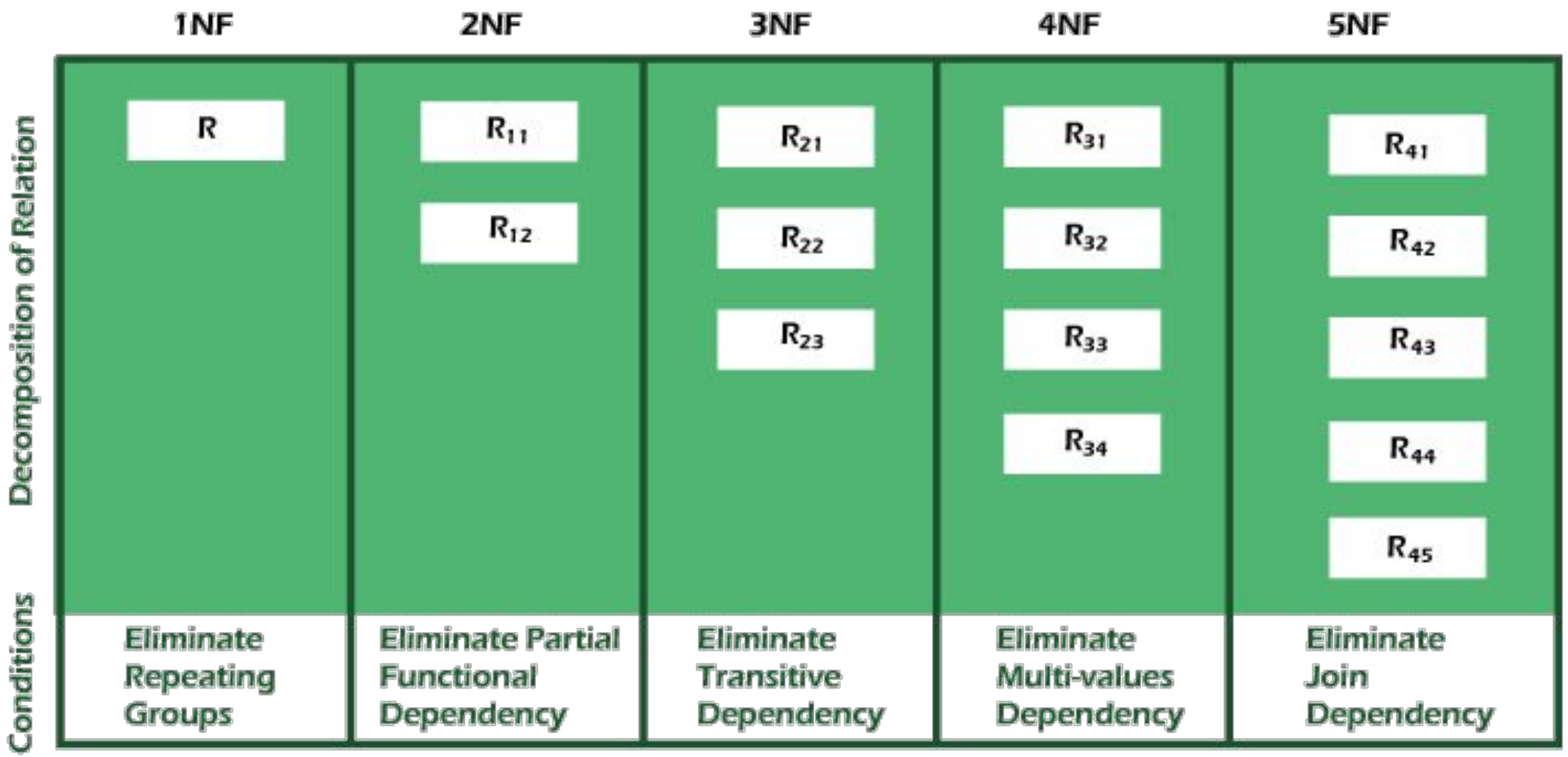
What is normalization?

- Normalization is the process of organizing the data in the database.
- Used to minimize the redundancy from a relation or set of relations.
- Used to eliminate undesirable characteristics like Insertion, Update, and Deletion Anomalies.
- Normalization divides the larger table into smaller and links them using relationships.
- The normal form is used to reduce redundancy from the database table.
- Normalization is a process of decomposing the relations into relations with fewer attributes.

Why normalization?

- As previously stated, normalization is used to eliminate data redundancy. It offers a mechanism for removing the following anomalies from the database and making it more consistent:
- A database anomaly is a fault in the database caused by insufficient preparation and redundancy.
- **Insertion anomalies** occur when we are unable to insert data into a database due to the absence of particular attributes at the moment of insertion.
- **Updation anomalies** occur when the same data items with the same values are repeated but are not related to each other.
- A **deletion anomaly** happens when deleting one part of the data results in the deletion of the other necessary information from the database.

Following are the various types of Normal forms:



Normal Form	Description
1NF	A relation is in 1NF if it contains an atomic value.
2NF	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.
3NF	A relation will be in 3NF if it is in 2NF and no transition dependency exists.
BCNF	A stronger definition of 3NF is known as Boyce Codd's normal form.
4NF	A relation will be in 4NF if it is in Boyce Codd's normal form and has no multi-valued dependency.
5NF	A relation is in 5NF. If it is in 4NF and does not contain any join dependency, joining should be lossless.

Advantages of Normalization

- Normalization helps to minimize data redundancy.
- Greater overall database organization.
- Data consistency within the database.
- Much more flexible database design.
- Enforces the concept of relational integrity.

Disadvantages of Normalization

- You cannot start building the database before knowing what the user needs.
- The performance degrades when normalizing the relations to higher normal forms, i.e., 4NF, 5NF.
- It is very time-consuming and difficult to normalize relations of a higher degree.
- Careless decomposition may lead to a bad database design, leading to serious problems.

First normal form (1NF)

- A relation will be 1NF if it contains an atomic value.
- It states that an attribute of a table cannot hold multiple values. It must hold only single-valued attribute.
- First normal form does not allow the multi-valued attribute, composite attribute and their combinations.

- Relation EMPLOYEE is not in 1NF because of multi-valued attribute EMP_PHONE.

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
14	John	7272826385, 9064738238	UP
20	Harry	8574783832	Bihar
12	Sam	7390372389, 8589830302	Punjab

- The decomposition of the EMPLOYEE table into 1NF has been shown below:

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
14	John	7272826385	UP
14	John	9064738238	UP
20	Harry	8574783832	Bihar
12	Sam	7390372389	Punjab
12	Sam	8589830302	Punjab

Second Normal Form (2NF)

- In the 2NF, relational must be in 1NF.
- In the second normal form, all non-key attributes are fully functional dependent on the primary key
  - Let's assume, a school can store the data of teachers and the subjects they teach. In a school, a teacher can teach more than one subject.

TEACHER_ID	SUBJECT	TEACHER_AGE
25	Chemistry	30
25	Biology	30
47	English	35
83	Math	38
83	Computer	38

  - In the given table, non-prime attribute TEACHER_AGE is dependent on TEACHER_ID which is a proper subset of a candidate key. That's why it violates the rule for 2NF.

To convert the given table into 2NF, we decompose it into two tables:

TEACHER_ID	TEACHER_AGE
25	30
47	35
83	38

TEACHER_ID	SUBJECT
25	Chemistry
25	Biology
47	English
83	Math
83	Computer

Third normal form (3NF)

- A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.
- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.
- A relation is in third normal form if it holds atleast one of the following conditions for every non-trivial function dependency $X \rightarrow Y$.
- X is a super key.
- Y is a prime attribute, i.e., each element of Y is part of some candidate key.

EMPLOYEE_DETAIL table:

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
222	Harry	201010	UP	Noida
333	Stephan	02228	US	Boston
444	Lan	60007	US	Chicago
555	Katharine	06389	UK	Norwich

- Super key in the table is,

{EMP_ID}, {EMP_ID, EMP_NAME},....so on

- **Candidate key:** {EMP_ID}
- **Non-prime attributes:** In the given table, all attributes except EMP_ID are non-prime.

EMP_ID	EMP_NAME	EMP_ZIP
222	Harry	201010
333	Stephan	02228
444	Lan	60007
555	Katharine	06389
666	John	462007

EMP_ZIP	EMP_STATE	EMP_CITY
201010	UP	Noida
02228	US	Boston
60007	US	Chicago
06389	UK	Norwich
462007	MP	Bhopal

Boyce Codd normal form (BCNF)

- BCNF is the advance version of 3NF. It is stricter than 3NF.
- A table is in BCNF if every functional dependency X → Y, X is the super key of the table.
- For BCNF, the table should be in 3NF, and for every FD, LHS is super key.
  - **Example:** Let's assume there is a company where employees work in more than one department.

EMP_ID	EMP_COUNTRY	EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
264	India	Designing	D394	283
264	India	Testing	D394	300
364	UK	Stores	D283	232
364	UK	Developing	D283	549

  - **In the above table Functional dependencies are as follows:**
  - EMP_ID → EMP_COUNTRY
  - EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}

- **EMP_COUNTRY table:**

EMP_ID	EMP_COUNTRY
264	India
364	India

- **EMP_DEPT table:**

EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
Designing	D394	283
Testing	D394	300
Stores	D283	232
Developing	D283	549

- **EMP_DEPT_MAPPING table:**

EMP_ID	EMP_DEPT
D394	283
D394	300
D283	232
D283	549

Fourth normal form (4NF)

- A relation will be in 4NF if it is in Boyce Codd normal form and has no multi-valued dependency.
- For a dependency $A \rightarrow B$, if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.

STUDENT

STU_ID	COURSE	HOBBY
21	Computer	Dancing
21	Math	Singing
34	Chemistry	Dancing
74	Biology	Cricket
59	Physics	Hockey

- The given STUDENT table is in 3NF, but the COURSE and HOBBY are two independent entity.
- Hence, there is no relationship between COURSE and HOBBY.
- In the STUDENT relation, a student with STU_ID, **21** contains two courses, **Computer** and **Math** and two hobbies, **Dancing** and **Singing**.
- So there is a Multi-valued dependency on STU_ID, which leads to unnecessary repetition of data.
- So to make the above table into 4NF, we can decompose it into two tables:

STUDENT_COURSE

STU_ID	COURSE
21	Computer
21	Math
34	Chemistry
74	Biology
59	Physics

STUDENT_HOBBY

STU_ID	HOBBY
21	Dancing
21	Singing
34	Dancing
74	Cricket
59	Hockey

Fifth normal form (5NF)

- A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.

- 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.

- 5NF is also known as Project-join normal form (PJ/NF).

SUBJECT	LECTURER	SEMESTER
Computer	Anshika	Semester 1
Computer	John	Semester 1
Math	John	Semester 1
Math	Akash	Semester 2
Chemistry	Praveen	Semester 1

- In the above table, John takes both Computer and Math class for Semester 1 but he doesn't take Math class for Semester 2.
- In this case, combination of all these fields required to identify a valid data.
- Suppose we add a new Semester as Semester 3 but do not know about the subject and who will be taking that subject so we leave Lecturer and Subject as NULL.
- But all three columns together acts as a primary key, so we can't leave other two columns blank.
- So to make the above table into 5NF, we can decompose it into three relations P1, P2 & P3:

P1

SEMESTER	SUBJECT
Semester 1	Computer
Semester 1	Math
Semester 1	Chemistry
Semester 2	Math

P2

SUBJECT	LECTURER
Computer	Anshika
Computer	John
Math	John
Math	Akash
Chemistry	Praveen

P3

SEMSTER	LECTURER
Semester 1	Anshika
Semester 1	John
Semester 1	John
Semester 2	Akash
Semester 1	Praveen